

Practical No: 08

Creating and comparing different text representations.

a) Write a program to create a bag of words (bow) text representation.

```
In [4]: import nltk
import numpy as np
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
```

```
Out[4]: True
```

```
In [5]: texts = [
    "The cat sat on the mat",
    "The dog sat on the log"
]

# Tokenize the texts
tokenized_texts = [nltk.word_tokenize(text.lower()) for text in texts]

# Create a vocabulary (set of all unique words)
vocabulary = sorted(set(word for text in tokenized_texts for word in text))
print("Vocabulary:", vocabulary)
```

```
Vocabulary: ['cat', 'dog', 'log', 'mat', 'on', 'sat', 'the']
```

```
In [6]: # Bag of Words (BoW) representation
def get_bow_representation(tokens, vocabulary):
    return [tokens.count(word) for word in vocabulary]

bow_vectors = [get_bow_representation(text, vocabulary) for text in tokenized_texts]

# Print BoW vectors
print("BoW vectors:")
print(np.array(bow_vectors))
```

```
BoW vectors:
[[1 0 0 1 1 1 2]
 [0 1 1 0 1 1 2]]
```

b) Write a program to create tf_idf text representations.

```
In [7]: import nltk
import numpy as np
from collections import Counter
```

```
from math import log
nltk.download('punkt')
```

[nltk_data] Downloading package punkt to /root/nltk_data...

[nltk_data] Package punkt is already up-to-date!

Out[7]: True

```
In [8]: texts = [
        "The cat sat on the mat",
        "The dog sat on the log"
    ]

    # Tokenize the texts
    tokenized_texts = [nltk.word_tokenize(text.lower()) for text in texts]

    # Create a vocabulary (set of all unique words)
    vocabulary = set(word for text in tokenized_texts for word in text)
    print("Vocabulary:", vocabulary)
```

Vocabulary: {'mat', 'on', 'cat', 'dog', 'the', 'log', 'sat'}

```
In [9]: # Function to compute Term Frequency (TF)
def get_tf(tokens, vocabulary):
    tf_vector = [tokens.count(word) for word in vocabulary]
    print("\nTF vectors:")
    print(tf_vector)
    return tf_vector

# Function to compute Inverse Document Frequency (IDF)
def get_idf(vocabulary, docs):
    num_docs = len(docs)
    idf_vector = []
    for word in vocabulary:
        # Count the number of documents containing the word
        num_docs_with_word = sum(1 for doc in docs if word in doc)
        # Calculate IDF as Log(num_docs / (1 + num_docs_with_word)) to avoid div
        idf_value = log(num_docs / (1 + num_docs_with_word)) + 1 # Adding 1 to
        idf_vector.append(idf_value)
    return idf_vector # Fix indentation to return after loop completes

# Function to compute TF-IDF
def get_tfidf(tokens, vocabulary, idf_vector):
    tf_vector = get_tf(tokens, vocabulary)
    tfidf_vector = [tf * idf for tf, idf in zip(tf_vector, idf_vector)]
    return tfidf_vector

# Calculate IDF for the entire corpus
idf_vector = get_idf(vocabulary, tokenized_texts)
print("\nIDF vectors:")
print(idf_vector)
```

IDF vectors:

[1.0, 0.5945348918918356, 1.0, 1.0, 0.5945348918918356, 1.0, 0.5945348918918356]

```
In [10]: # Compute TF-IDF for each document
tfidf_vectors = [get_tfidf(text, vocabulary, idf_vector) for text in tokenized_t

# Print TF-IDF vectors
print("\nTF-IDF vectors:")
print(np.array(tfidf_vectors))
```

TF vectors:
[1, 1, 1, 0, 2, 0, 1]

TF vectors:
[0, 1, 0, 1, 2, 1, 1]

TF-IDF vectors:
[[1. 0.59453489 1. 0. 1.18906978 0.
0.59453489]
[0. 0.59453489 0. 1. 1.18906978 1.
0.59453489]]

c) Write a program to compare two vectors of bow using cosine similarity.

```
In [11]: import nltk
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

Out[11]: True

```
In [12]: texts = [
    "The cat sat on the mat",
    "The dog sat on the log"
]

# Tokenize the texts
tokenized_texts = [nltk.word_tokenize(text.lower()) for text in texts]

# Create a vocabulary (set of all unique words)
vocabulary = set(word for text in tokenized_texts for word in text)
print(vocabulary)

{'mat', 'on', 'cat', 'dog', 'the', 'log', 'sat'}
```

```
In [13]: # Bag of Words (BoW) representation
def get_bow_representation(tokens, vocabulary):
    return [tokens.count(word) for word in vocabulary]
bow_vectors = [get_bow_representation(text, vocabulary) for text in tokenized_texts]

# Print BoW vectors
print("BoW vectors:")
print(np.array(bow_vectors))

bow_similarity = cosine_similarity([bow_vectors[0]], [bow_vectors[1]])[0][0]
print(bow_similarity)
```

BoW vectors:
[[1 1 1 0 2 0 1]
[0 1 0 1 2 1 1]]
0.7499999999999999

```
In [ ]: ### d) Write a program to compare a bow vector with a tf-idf vector using cosine
```

```
In [14]: import nltk
import numpy as np
```

```

from collections import Counter
from math import log
from sklearn.metrics.pairwise import cosine_similarity

```

```

In [15]: # Ensure you have the necessary NLTK resources
nltk.download('punkt')
texts = [
    "The cat sat on the mat",
    "The dog sat on the log"
]
# Tokenize the texts
tokenized_texts = [nltk.word_tokenize(text.lower()) for text in texts]
# Create a vocabulary (set of all unique words)
vocabulary = set(word for text in tokenized_texts for word in text)
print("Vocabulary:", vocabulary)

```

Vocabulary: {'mat', 'on', 'cat', 'dog', 'the', 'log', 'sat'}

[nltk_data] Downloading package punkt to /root/nltk_data...

[nltk_data] Package punkt is already up-to-date!

```

In [16]: # Bag of Words (BoW) representation
def get_bow_representation(tokens, vocabulary):
    return [tokens.count(word) for word in vocabulary]

bow_vectors = [get_bow_representation(text, vocabulary) for text in tokenized_texts]

# Function to compute Term Frequency (TF)
def get_tf(tokens, vocabulary):
    return [tokens.count(word) for word in vocabulary]

def get_idf(vocabulary, docs):
    num_docs = len(docs)
    idf_vector = []
    for word in vocabulary:
        # Count the number of documents containing the word
        num_docs_with_word = sum(1 for doc in docs if word in doc)
        # Calculate IDF as log(num_docs / (1 + num_docs_with_word)) to avoid div
        idf_value = log(num_docs / (1 + num_docs_with_word)) + 1 # Adding 1 to
        idf_vector.append(idf_value) # ✅ Collect all values
    return idf_vector # ✅ Return the full vector **after** the loop

# Function to compute TF-IDF
def get_tfidf(tokens, vocabulary, idf_vector):
    tf_vector = get_tf(tokens, vocabulary)
    tfidf_vector = [tf * idf for tf, idf in zip(tf_vector, idf_vector)]
    return tfidf_vector

# Calculate IDF for the entire corpus
idf_vector = get_idf(vocabulary, tokenized_texts)
# print("\nIDF vectors:")
# print(idf_vector)

# Compute TF-IDF for each document
tfidf_vectors = [get_tfidf(text, vocabulary, idf_vector) for text in tokenized_texts]

# Compute cosine similarity between BoW and TF-IDF vectors for doc1
bow_similarity = cosine_similarity([bow_vectors[0]], [tfidf_vectors[0]])[0][0]
print("\nCosine Similarity between doc1 (BoW) and doc1 (TF-IDF):", bow_similarity)

```

```
# Compute cosine similarity between BoW and TF-IDF vectors for doc2  
bow_similarity = cosine_similarity([bow_vectors[1]], [tfidf_vectors[1]])[0][0]  
print("Cosine Similarity between doc2 (BoW) and doc2 (TF-IDF):", bow_similarity)
```

Cosine Similarity between doc1 (BoW) and doc1 (TF-IDF): 0.9696169252036222

Cosine Similarity between doc2 (BoW) and doc2 (TF-IDF): 0.9696169252036222